Lesson Plan – SDEV2801 – Technical Documentation and Facilitation

Lesson 10 – Understanding Technical Documentation: Purpose, Audience, and Types

Pre-lesson reflection:
I plan to remove barriers to learning by using inclusive teaching strategies and educational best practices. This includes using clear and consistent instructions, providing materials in multiple formats (text, audio, visual), incorporating Universal Design for Learning (UDL) principles, and creating a supportive classroom environment where diverse perspectives are valued. I will also check in regularly to identify individual learning needs and adjust as needed to support all students in achieving success.

LESSON OBJECTIVE: PARTICIPANTS SHOULD BE ABLE TO EXPLAIN THE PURPOSE OF TECHNICAL DOCUMENTATION IN SOFTWARE DEVELOPMENT, IDENTIFY COMMON DOCUMENTATION TYPES, AND RECOGNIZE HOW AUDIENCE AND PURPOSE INFLUENCE DOCUMENTATION STRUCTURE, STYLE, AND TOPIC SELECTION.

RELATED COURSE OUTCOME: BY THE END OF THIS COURSE, PARTICIPANTS SHOULD BE ABLE TO:

2. Conduct research and write technical documentation that conveys complex information.

Resources Required:

- **Slide deck**
- **Whiteboard or shared digital space**
- **Technical Documentation and Presentation Assessment specs and rubric**
- **Activity scenarios (handout, slide, send to breakout rooms in text – depends on delivery mode)**

Time: 110 minutes

Instructional Event	Time	Content	Activity
CONNECTING **MOTIVATE COGNITIVE PRESENCE:** • **Motivate learner attention** • **Stimulate recall of prior learning** • **Inform learner of objective(s)**	30 min	Review and Introduction/Connecting Activity to Today's Lesson	• We are moving today from Module 1 (Facilitated Discussions) to Module 2 (Technical Documentation & Technical Presentations) • Go through Technical Document and Individual Technical Presentation specs and rubrics on Brightspace. • Today's lesson connecting activity: • Ask students to think of a time they had to figure out a new concept or how to put something together or use a new technology without guidance. (e.g. a programming framework, a software tool, an API, a new application). • Have them share/discuss: ○ Where did you look for help? • Was the help actually helpful? What made it good or bad? **Key Transition - Every piece of software that survives in industry depends on good documentation. Without it, developers cannot maintain systems, users cannot adopt tools, and teams cannot collaborate effectively.**
EXPLORING **INSTRUCT AND DEMONSTRATE:**	40 min	Mini-lesson: Understanding Technical Documentation (see below)	n/a

• **Present new information** • **Integrate learner participation**			
PRACTICING **PROVIDE OPPORTUNITIES FOR GUIDED PRACTICE:** • **Coach learners through relevant applications of content** • **Provide feedback on performance**	30 min	See next column	Perform Technical Documentation Activity (see below)
ASSESSING **ASSESS THE LEARNING OF THE OBJECTIVE(S)** • **Assess individual learning** • **Provide feedback on performance** • **Elicit feedback on the learning**	10 min	See next column	Have students start brainstorming topic ideas for their upcoming assessment. Encourage them to share with you in class or send to you outside of class to help guide them.
REFLECTING **REFLECT ON WHAT WORKED AND DIDN'T WORK AND NOTE REVISIONS FOR NEXT TIME**			

OpenAI. (2026). ChatGPT-generated instructional resources for the module objective: Analyze and apply individual and group dynamic organizational behaviour principles and theories [Large language model]. ChatGPT. https://chat.openai.com/

Understanding Technical Documentation

Introduction:

- Before you begin writing any technical document, it is important to understand what technical documentation actually is and why it plays such a critical role in the software industry.
- Software developers are often expected to do more than just write code. They must also explain systems, document processes, guide users, and share knowledge with other developers and stakeholders.
- Clear documentation allows teams to collaborate effectively, helps new developers understand existing systems, and ensures that software can be maintained and improved over time.
- In this lesson, we will look at:
    - What technical documentation is.
    - The purposes it serves in software development.
    - Different types of documents commonly used in the industry.
    - How the intended audience influences the way technical information is written and organized.
- Understanding these foundations will help you make better decisions when planning and writing your own technical document for the upcoming assessment.

What is Technical Documentation?

- Technical documentation refers to written materials that explain how a system works, how to use it, or how to build, maintain, or troubleshoot it. In software development, documentation is considered an essential component of the product itself rather than an optional supplement.
- Modern software systems are rarely built or maintained by a single person. Teams collaborate across roles such as developers, testers, project managers, and clients. Technical documentation acts as a shared knowledge resource that allows these groups to communicate effectively and maintain continuity over time.
- For example, when a developer joins a new team, documentation helps them quickly understand the architecture of the system, the development environment, and coding conventions. Similarly, end users rely on documentation to understand how to install, configure, and use software tools.
- Technical documentation typically serves three key functions:
    - Knowledge transfer - documentation allows knowledge to persist beyond individual team members. When developers leave or change roles, documentation preserves institutional knowledge.
    - Operational guidance - documentation provides instructions that help people complete tasks, such as installing software, configuring systems, or using features.
    - System understanding - technical documents explain how systems work internally so that developers can maintain, extend, and debug them.
    - Defining system requirements and expectations - in many software projects, developers work with clients or stakeholders to create requirements documents that outline system features, constraints, and expectations. These documents help ensure that everyone involved in the project shares a clear understanding of what will be developed and how success will be measured.
- In software development environments, documentation is often integrated directly into development workflows. Examples include:
    - API documentation generated from code comments
    - README files in Git repositories

    - Architecture documentation describing system components
    - Deployment guides for DevOps environments
- Strong documentation improves team efficiency and reduces errors. Poor documentation, on the other hand, can slow development, increase onboarding time, and make systems harder to maintain.
- *Instructor note – consider adding short interaction with students here to keep them engaged – for example:*
    - *Remind students of the three functions of technical documentation - Knowledge transfer, Operational guidance, System understanding*
    - *Read a short example and ask students to identify which function of technical documentation it represents:*
        - *"Step-by-step instructions for installing Docker on Windows." (answer should be operational guidance)*
        - *"Architecture documentation explaining how microservices communicate." (answer should be system understanding)*
        - *"A README explaining how a project is structured so new developers can contribute." (answer should be knowledge transfer)*

Purpose of Technical Documentation:

- Technical documentation is created to help people understand, use, build, or maintain technology.
- While the specific format of documentation may vary, its purpose usually falls into several broad categories.
    - Explanation:
        - Some documents exist to explain how a technology or concept works. These documents often focus on describing architecture, processes, or design decisions.
        - Example: A document explaining how REST APIs work and how they structure communication between services.
        - These types of documents often appear as technical overviews, architecture documentation, or concept explanations.
    - Instruction:
        - Instructional documentation provides step-by-step guidance for completing tasks. These documents are designed to help users accomplish specific goals with minimal confusion.
        - Examples include:
            - Installation guides
            - Setup instructions
            - Tutorials
            - Configuration documentation
        - These documents emphasize **clarity, sequence, and accuracy** because users depend on them to perform tasks successfully.
    - Support and Troubleshooting:
        - Technical documentation also supports users and developers when problems occur. Troubleshooting guides help readers diagnose issues and identify solutions.
        - Examples include:
            - Error message documentation
            - Debugging guides

        - FAQ sections
        - Known issues lists
    - These documents reduce support requests and allow users to resolve issues independently.
- Standardization, Specifications, and Consistency
    - Technical documentation also helps teams establish shared specifications, requirements, and processes. These documents ensure that everyone involved in a project understands what must be built, how systems should behave, and how teams should work together.
    - Examples include:
        - Software requirements documents
        - Technical specifications
        - Coding standards
        - Development workflows
        - Deployment procedures
    - These documents ensure that teams share a clear and consistent understanding of system expectations and development practices.

- *Instructor note – consider adding short interaction with students here to keep them engaged – for example:*
    - *Ask students for each statement "is this an explanation, instruction, support/troubleshooting, or standardization/specifications form of technical documentation?"*
        - *A document explaining how microservices communicate through an API gateway. (explanation)*
        - *A guide showing step-by-step instructions for installing Docker. (instruction)*
        - *A page listing common error messages and how to fix them. (support/troubleshooting)*
        - *A document outlining coding standards that all developers will follow (standardization/specifications)*

Consider the Audience:

- One of the most important decisions in technical communication is identifying the intended audience. The audience determines how information should be organized, how much background knowledge should be assumed, and what level of technical language is appropriate.
- Technical documents are rarely written for a generic audience. Instead, they are designed for specific groups of readers who have particular goals and levels of expertise.
- Consider common audiences in software development (these are not the only audiences, but are common ones you will encounter):
    - End Users:
        - End users typically interact with software but do not need to understand how it is implemented internally. Documentation for this audience should prioritize clarity and simplicity.
            - Key characteristics of user-focused documentation include:
            - Minimal technical jargon
            - Step-by-step instructions
            - Visual aids such as screenshots
            - Focus on completing tasks

        - Example of end user documentation - a user manual explaining how to create a new account in a budgeting application.
    - Junior Developers:
        - Documentation written for junior developers assumes some technical background but may still include explanations and examples.
        - These documents often include:
            - Definitions of technical terms
            - Code examples
            - Explanations of design decisions
        - Example - a guide explaining how to integrate an authentication library into a web application.
    - Experienced Developers:
        - Highly technical audiences typically prefer concise documentation that focuses on implementation details rather than introductory explanations.
        - Documentation for experienced developers often includes:
            - API references
            - configuration details
            - architecture diagrams
            - technical specifications
        - Example - documentation describing the endpoints, parameters, and response formats for a REST API.
    - Clients:
        - Clients are stakeholders who commission, fund, or request the development of a software system. They are typically interested in understanding what the system does, how it supports their goals, and how it will be implemented or delivered, but they may not have deep technical expertise. Documentation for clients should balance clarity with enough detail to explain the system's capabilities and decisions.
        - Documentation written for clients often includes:
            - Clear explanations of system features and capabilities
            - High-level descriptions of how the system works
            - Diagrams or visuals that illustrate workflows or architecture at a conceptual level
            - Explanations of project decisions, scope, or requirements
        - Example - a system overview explaining how a budgeting application stores transactions, categorizes spending, and generates reports for users.
    - System Administrators/IT Operations:
        - System administrators are responsible for installing, configuring, maintaining, and monitoring software systems after they are deployed. They may not be the developers who built the system, but they require technical documentation to ensure the system runs reliably in production environments.
        - Documentation for system administrators often includes:
            - Installation and deployment instructions
            - Configuration settings and environment requirements
            - Troubleshooting and maintenance procedures
            - Security and access control guidelines

        - Example - documentation explaining how to deploy a web application to a server environment, configure environment variables, and monitor system logs.
- The same system may require multiple types of documentation because different audiences need different information. Effective technical writers must adapt their explanations, vocabulary, and structure to suit the needs of their readers.
- *Instructor note – consider adding short interaction with students here to keep them engaged – for example:*
    - *Who is the audience based on the following sentences?*
        - *"Click Create Account, enter your email and password, and follow the prompts to finish registration." (end users)*
        - *"Install the authentication package using npm install auth-lib, then configure the middleware in your Express application." (junior developers)*
        - *"The API returns a JSON response with a JWT token in the authorization header. Requests must include the token for subsequent authenticated endpoints." (experienced developers)*
    - *OR could do the following:*
        - *Show two versions of the same instruction – which one is better for an end user?*
            - *A - Execute the authentication module to generate a user token.*
            - *B - Click Log In and enter your username and password.*
            - *(answer should be B)*

Types of Technical Documentation:

- Technical documentation can take many forms depending on the purpose of the document and the needs of the audience. In software development, several common types appear frequently (this is not an exhaustive list):
    - Technical Explanations:
        - These documents describe how a concept or system works. They are often used to introduce technologies or explain design decisions.
        - Example: "How Microservices Architecture Works"
        - These documents emphasize conceptual understanding rather than step-by-step instructions.
    - Developer Guides:
        - Developer guides help programmers implement features or integrate tools into their systems.
        - These documents balance explanation with practical instruction.
    - User manuals:
        - User manuals focus on helping end users understand how to operate software systems. They often include instructions, screenshots, and troubleshooting advice.
        - These documents prioritize clarity and usability.
    - Installation and Setup Guides
        - These documents help users configure or install software systems.
        - Example: "Installing Docker on Windows"
    - Requirements Documents
        - A requirements document describes what a system must do and the conditions under which it must operate. Unlike a user manual or tutorial, which explains how to use a

system after it has been built, a requirements document is typically created before development begins.
    - Its primary purpose is to ensure that developers, clients, project managers, and testers share the same understanding of the system being built.
    - We will explore this type of technical document in more detail in the second half of the course when you develop a requirements document.

Conclusion:

- Technical documentation plays a critical role in the software development process. It allows teams to share knowledge, guide users, explain systems, and ensure that software can be developed, maintained, and improved over time. While developers are often known for writing code, the ability to clearly document technical information is an equally important professional skill.
- Throughout this lecture, we looked at what technical documentation is, the different purposes it serves, and the variety of document types used in the software industry. We also discussed how identifying the audience and purpose of a document helps determine the appropriate structure, level of detail, and tone.
- Whether the goal is to explain a concept, guide users through a process, define system requirements, or support troubleshooting, effective documentation must always be designed with the reader in mind.
- As you begin thinking about your upcoming assessment, remember that strong technical documents do more than present information. They organize complex ideas clearly, anticipate the needs of the audience, and communicate technical concepts in a way that helps others understand and act on the information provided.

OER Resources:

Google. (2022). *Technical writing courses for engineers*. Google Open Source Education. https://developers.google.com/tech-writing

Hart-Davidson, W., & Gentle, A. (2017). *Developing quality technical information: A handbook for writers and editors* (Open edition). IBM Press. https://www.ibm.com/developerworks/library/tech-technical-writing/

Open Technical Communication. (2021). *Technical communication*. BCcampus Open Education. https://opentextbc.ca/technicalcommunication/

Redish, J. (2012). *Letting go of the words: Writing web content that works* (Open access selections). Morgan Kaufmann. https://www.redish.net/writing-for-the-web/

Spinuzzi, C. (2020). *Technical communication and workplace writing* (Open educational materials). University of Texas. https://writingspaces.org/

Technical Documentation Activity

Identify the Purpose, Audience, Type, and Headings for a Scenario:

- Students work in **small groups (3–4 students)**. Provide each group with **a sample scenario** such as:
  - Scenario 1 - A company releases a new budgeting app for consumers.
  - Scenario 2 - A development team is implementing OAuth authentication in a web application.
  - Scenario 3 - A company wants documentation explaining how Git branching works for junior developers.
  - Scenario 4 - A company has developed a new mobile fitness tracking app. Many new users are struggling to understand how to log workouts and track their progress.
  - Scenario 5 - A development team is adopting Docker for the first time and needs documentation explaining how to install Docker and run the project locally.
  - Scenario 6 - A software company wants to explain how their internal microservices communicate with each other so new developers can understand the system architecture.
- Group Tasks – for their assigned scenario, groups must decide:
  - What is the purpose of the technical document?
    - Explanation
    - Instruction
    - Support and troubleshooting
    - Standardization, specifications, and consistency
  - Who is the primary audience?
  - What type of technical documentation is required?
    - Technical explanation
    - Developer guide
    - User Manual
    - Installation and Setup guide
    - Requirements document
    - Other
  - What could 3-5 headings be for the document?
- Debrief:
  - Each group briefly shares (on digital whiteboard or through discussion) what they decided.

Instructor resources (sample answers to guide you):

Scenario 1
A company releases a new budgeting app for consumers.

Purpose of the Technical Document
Instruction
(The goal is to guide users on how to use the application and complete tasks within it.)
Primary Audience
End users / consumers using the budgeting app who may have little or no technical background.
Type of Technical Documentation Required
User Manual (User Guide)

Possible Headings for the Document

- Introduction and Overview of the App
- Creating an Account and Logging In
- Setting Up Your First Budget
- Tracking Income and Expenses
- Viewing Reports and Budget Insights

Scenario 2

A development team is implementing OAuth authentication in a web application.

Purpose of the Technical Document

Instruction

(The goal is to guide developers on how to implement OAuth authentication within the application.)

Primary Audience

Developers (likely junior or intermediate developers responsible for implementing authentication features in the application).

Type of Technical Documentation Required

Developer Guide (Technical Implementation Guide)

Possible Headings for the Document

- Introduction to OAuth Authentication
- Prerequisites and Required Libraries
- OAuth Authentication Flow
- Implementing OAuth in the Application
- Security Considerations and Best Practices

Scenario 3

A company wants documentation explaining how Git branching works for junior developers.

Purpose of the Technical Document

Explanation

(The goal is to help readers understand the concept of Git branching and how it is used in development workflows.)

Primary Audience

Junior developers who are learning how version control workflows function within a development team.

Type of Technical Documentation Required

Technical Explanation (could also be structured as a Developer Guide)

Possible Headings for the Document

- Introduction to Git Branching
- Why Branching Is Used in Software Development
- Common Branch Types (main, feature, hotfix)
- Example Git Branching Workflow
- Best Practices for Managing Branches

Scenario 4
A company has developed a new mobile fitness tracking app. Many new users are struggling to understand how to log workouts and track their progress.

Purpose of the Technical Document
Instruction
(The goal is to guide users on how to perform key tasks within the app.)
Primary Audience
End users / consumers using the fitness tracking app who may have little or no technical background.
Type of Technical Documentation Required
User Manual (User Guide)
Possible Headings for the Document

- Introduction and Overview of the App
- Creating an Account and Setting Up Your Profile
- Logging and Managing Workouts
- Tracking Progress and Viewing Statistics
- Troubleshooting Common Issues

Scenario 5
A development team is adopting Docker for the first time and needs documentation explaining how to install Docker and run the project locally.

Purpose of the Technical Document
Instruction
(The goal is to guide developers through installing Docker and configuring their environment so the project can run locally.)
Primary Audience
Developers on the team, likely junior or new developers who need to set up the development environment.
Type of Technical Documentation Required
Installation and Setup Guide
Possible Headings for the Document

- Introduction and Purpose of the Guide
- System Requirements and Prerequisites
- Installing Docker
- Pulling the Project Repository
- Running the Application Locally and Verifying Installation

Scenario 6
A software company wants to explain how their internal microservices communicate with each other so new developers can understand the system architecture.

Purpose of the Technical Document
Explanation
(The goal is to help developers understand how the system architecture works and how the microservices interact.)

Primary Audience

Junior developers or new developers joining the team who need to understand the system architecture.

Type of Technical Documentation Required

Technical Explanation (Architecture Overview)

Possible Headings for the Document

- Introduction to the System Architecture
- What Are Microservices?
- How Services Communicate Within the System
- Role of the API Gateway and Service Communication
- Example Workflow of a Request Through the System